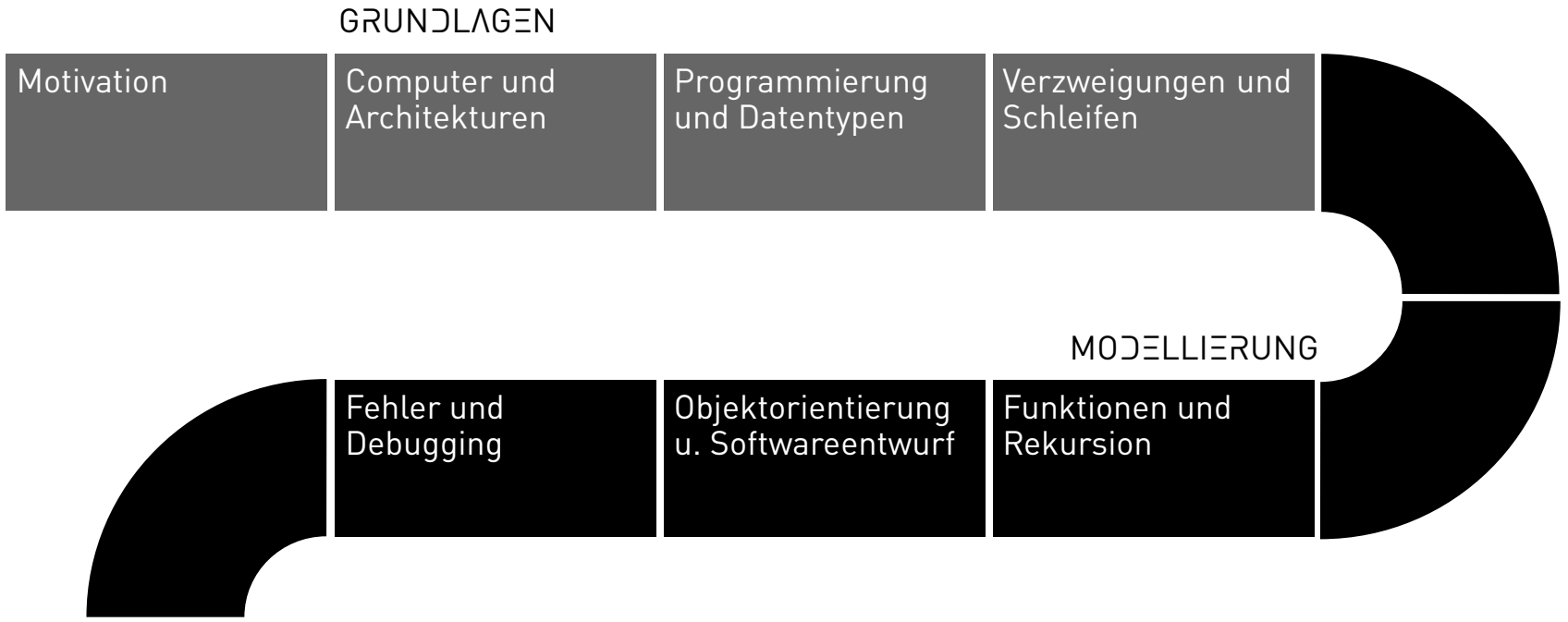
Programming and Databases

Joern Ploennigs

Loops

MIDJOURNEY: LOOPS, REF. ROBERT DELAUNAY

Process


GRUNDLAGEN
Motivation
Computer und Architekturen
Programmierung und Datentypen
Verzweigungen und Schleifen
MODELLIERUNG
Funktionen und Rekursion
Objektorientierung u. Softwareentwurf
Fehler und Debugging

Loops

Loops are an important approach in programming to implement repetitive or incremental solutions.
In Python, the following are available:

- `for` (equivalent to a for-each loop)
- `while` (equivalent to a while loop)

Important properties:

- They work exactly like all other control statements, using indentation for the block that follows the header line
- Unlike in functions, loop variables are also accessible outside the loop!
- The other loop variants are not explicitly present, but can be functionally emulated

For Loops - FOR Loop

The for loop repeats a block of statements for all elements in a sequence. Element is a variable that is always bound to the current element from the sequence.

```
for Element in Sequenz:
    # Statement
```

Loops - For-each Loop Example

The for loop in Python is a for-each loop. It always iterates through a sequence of values, such as a list. The iterator variable `e` contains the current value from the list `seq`.

```
seq = ['a','b','c']
for e in seq:
    print(e)
```

Output:

```
a
b
c
```

Loops - FOR-Loop Example

The for loop in Python can also be used as a traditional for loop, in which you repeat something n times. To do this, you generate a sequence of numbers with the `range(n)` function, through which the for-each loop then iterates.

```
n = 3
seq = range(n)
for e in seq:
    print(e)
```

Output:

```
0
1
2
```

Loops - FOR Loop Example

This can be used, for example, to convert a list of measurements from feet in the imperial system to metres.

```
measurements_feet = [4.2, 2.3, 6.2, 10.5]   # Input
measurements_meter = []   # Results

for feets in measurements_feet:
    measurements_meter.append(0.3048 * feets)

print(measurements_meter)
```

Output:

```
[1.2801600000000002, 0.70104, 1.88976, 3.2004]
```

Loops - WHILE Loop

The while loop runs as long as a condition is true. Since this is evaluated at the start and can be false from the outset, the body may not be executed.

```
while condition:
    # statement
```

As with an if statement, the condition is evaluated for its truth value:

- If this is `True`, the loop runs once, after which it is checked again
- If this is `False`, the loop ends and the code is not (re)executed

Loops - While Loop vs. Do-While Loop vs. Repeat-Until

While loop

In a while loop, the condition can be false right from the start. So the loop does not have to be executed.

```
Bedingung = True/False
while Bedingung:
    # Statement
    Bedingung = False
```

Do-While loop

In a do-while loop, the condition is checked only at the end. The loop is therefore always executed at least once.

```
Bedingung = True
while Bedingung:
    # Statement
    Bedingung = False
```

Repeat-Until loop

In a Repeat-Until loop, the condition is also checked only at the end. The loop is repeated until the condition becomes true.

```
Bedingung = False
while not Bedingung:
    # Statement
    Bedingung = True
```

Program Flow Control

We now know two tools:

Conditional execution:

- `if`, `else`, `elif`

Looping:

- `while`, `for`

Every program that has ever run on a computer could be written with these tools.

PROGRAMMIERUNG UND DATENBANKEN